



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Code of Conduct:

I Nithya Priya / 192421119 that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question and Answers

1. A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Answer:

Software testing should be treated as a planned and systematic engineering activity rather than random testing. The team should first prepare a test plan containing the testing objectives, scope, risks, resources, and schedule.

Test cases should be designed systematically from requirements. Techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing, and Risk-Based Testing can be used instead of ad-hoc testing.

Testing tools can be introduced for test management, defect tracking, automation, API testing, and performance testing. Standards such as ISO/IEC/IEEE 29119 can be followed to maintain proper testing processes and documentation.

For example, if a banking application allows fund transfers from ₹1 to ₹50,000, testers should check ₹1, ₹49,999, ₹50,000 and ₹50,001 instead of testing only normal amounts. This can identify boundary errors and invalid transactions.

Thus, systematic testing provides better test coverage, detects defects earlier, reduces repeated failures, and improves the reliability and quality of the banking application.

2. An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to

ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

Answer:

A structured testing process should be followed throughout the Software Development Life Cycle (SDLC) instead of testing only after the software is developed. A proper process makes testing consistent, repeatable, and effective.

The first stage is Test Planning. In this stage, the testing objectives, scope, resources, schedule, risks, and entry and exit criteria are defined. This ensures that all important ATM functions are included in testing.

The second stage is Test Analysis and Design. Requirements are studied and converted into test conditions and test cases. For an ATM, test cases should cover valid cards, incorrect PINs, withdrawal limits, insufficient balance, and invalid withdrawal amounts.

The third stage is Test Implementation. Required test data, test environment, cards, accounts, balances, and testing scripts are prepared before execution.

The fourth stage is Test Execution. Test cases are executed and actual results are compared with expected results. Any difference is recorded as a defect with proper details.

The next stage is Test Reporting and Evaluation. Testing progress, defects, coverage, and pass/fail results are analysed to decide whether the system is ready for release. Finally, during Test Closure, test documents and results are archived and lessons learned are recorded.

For ATM withdrawal functionality, the system should verify that a valid card and correct PIN are accepted, the requested amount is within the daily limit and available balance, the amount is dispensed correctly, and the account balance is updated properly. Unit testing can check withdrawal logic, integration testing can verify communication with the bank server, and system testing can verify the complete withdrawal process.

Following the same testing process for every release reduces defects because important activities are not skipped. Requirements remain traceable to test cases, defects are documented consistently, and regression testing ensures that previous problems do not return.

As a result, the ATM system will have more consistent results, fewer production defects, better reliability and security, reduced rework, and improved customer confidence.

3. A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Answer:

In software testing, error, defect, and failure are different concepts. Understanding these terms correctly helps testers identify problems and report them clearly.

An error is a human mistake made during requirements, design, coding, or other development activities. For example, a developer may misunderstand the requirement that an account should be locked after three incorrect login attempts.

A defect is the actual fault present in the software because of an error. In the above example, if the developer writes the condition incorrectly and allows four attempts instead of three, the incorrect condition becomes a defect.

A failure occurs when the defect is executed and the software produces an incorrect or unexpected result. If a user enters the wrong password three times and the account is still not locked, the user observes a failure.

Thus, the relationship can be understood as: Human Error → Defect in Software → Failure during Execution. However, every defect does not always produce a failure because the faulty part may not be executed under the required conditions.

Improper understanding of these terms can create confusion between testers and developers. It may also result in incorrect defect counts, poor root-cause analysis, and difficulty in fixing problems.

Defect reporting can be improved by using a standard format containing defect ID, title, module, environment, steps to reproduce, expected result, actual result, severity, priority, and screenshots or logs. The report should clearly describe what happened without unnecessarily assuming the cause.

For example, instead of writing “Login is not working,” a better report would state: *“Enter a valid username and password, but the application displays an invalid username message.”* This gives the developer clear information to reproduce and investigate the problem.

Therefore, clearly distinguishing errors, defects, and failures and maintaining structured defect reports improves communication, tracking, and the overall quality of the web application.

4. A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Answer:

The situation can be addressed using the software testing principle called defect clustering. This principle states that a small number of modules usually contain a large number of defects. Therefore, testing effort should be concentrated on high-risk and defect-prone modules. First, the team should analyse previous defect reports and identify the modules with the highest number of defects. These modules should receive additional test cases, detailed reviews, and more regression testing.

Test cases can be prioritised based on risk, previous defect history, business importance, recent changes, and requirement coverage. Critical functions should be tested before low-risk features, especially when testing time is limited.

Other testing principles should also be considered. Exhaustive testing is not possible, so important scenarios should be selected carefully. The pesticide paradox should also be

considered by regularly reviewing and updating test cases so that they continue to find new defects.

For example, in an online shopping application, suppose the payment module contains most of the previous defects. Testers should give higher priority to payment testing and perform additional boundary, integration, security, and regression tests on this module.

By applying defect clustering and risk-based prioritisation, testing effort is used more effectively. Important defects can be detected earlier, testing time is reduced, and release delays can be avoided.

Therefore, applying testing principles helps the team focus on areas where defects are most likely to occur and improves the overall efficiency and quality of testing.

5. A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Answer:

A defect repository is a central place where information about software defects is stored. It contains details such as the defect description, type, severity, affected module, root cause, status, and solution. The repository should not only be used for recording defects but also for learning from previous problems.

Defects can originate at different stages of software development. They may occur because of unclear requirements, incorrect design, programming mistakes, improper testing, or changes made during maintenance. Common causes include poor communication, lack of reviews, limited skills, and time pressure.

Defects can be classified according to their type, severity, development phase, affected module, and root cause. This classification helps the testing team identify common problem areas.

Repository data can be used to improve test design. For example, if previous defects show that many problems occurred because of incorrect input validation, testers can create more boundary value and invalid-input test cases. Previously discovered defects can also be added to the regression test suite.

To prevent recurring defects, every defect should contain proper root-cause information. The team should regularly analyse defect patterns, conduct review meetings, update coding and testing guidelines, and include important fixed defects in automated regression testing.

For example, if 15 out of 40 previous defects were caused by missing input validation, the next release should include validation checklists and additional tests for all important input fields.

6. A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Answer:

Crashes during fund transfer are serious because they can affect financial transactions and customer trust. The tester should first reproduce the crash and record important details such as the device, operating system, application version, network condition, and exact steps performed.

Crash logs, server logs, and API responses should then be analysed to identify the source of the problem. The tester should determine whether the problem is related to the user interface, business logic, API, or database.

Systematic testing should cover different transaction amounts, payees, network conditions, devices, and application states. Boundary values, invalid inputs, state transitions, and interruption scenarios should also be tested.

For example, suppose a user transfers ₹5,000 to a new payee using a weak network. During OTP verification, the request times out and the application crashes. The root cause may be an unhandled timeout exception in the OTP service. The solution could include proper timeout handling, retry mechanisms, and a suitable error message.

To prevent similar problems, the team should perform compatibility testing on different devices and operating systems, interruption testing, automated regression testing, performance testing, and code reviews focusing on exception handling.

Thus, systematic testing helps identify the actual root cause of crashes, improves application stability, and prevents previously fixed problems from occurring again.

7. A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Answer:

Testing and debugging are related but different activities in software development. Testing is used to identify failures and determine whether the software behaves according to the specified requirements. Debugging is the process of finding the cause of a failure and correcting the defect in the software.

Testing helps to discover problems, measure software quality, and check whether requirements are satisfied. Debugging helps developers locate the faulty code, correct it, and prevent similar problems through root-cause analysis.

For example, if a shopping application does not apply a discount correctly, the tester reports the failure with proper steps and expected and actual results. The developer then investigates the code, identifies the incorrect discount calculation, and fixes it.

Confusing testing with debugging can reduce testing effectiveness. A developer may fix only the reported case without checking other related cases. Similarly, if testers spend too much time trying to find the programming cause, other important tests may be delayed.

A proper process should be followed: first the tester executes the test and reports the failure, then the defect is analysed and assigned. The developer reproduces the problem, finds the root cause, and fixes it. After the fix, the tester performs retesting and regression testing to ensure that the correction works and has not created new defects.

Therefore, testing and debugging should work together but remain separate activities. This structured approach improves defect detection, correction, and overall software quality.

8. An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Answer:

Cross-browser testing is performed to ensure that a web application works correctly and consistently across different browsers, browser versions, operating systems, and devices.

Defects may occur because browsers use different rendering engines and may interpret HTML, CSS, and JavaScript differently. Differences in browser features, screen sizes, cookies, storage, fonts, and third-party plugins can also create problems.

The tester should first identify the browsers and devices commonly used by customers. Important functions such as login, search, shopping cart, checkout, and payment should receive higher priority. Test cases should then be executed across the selected browser and device combinations.

For example, a checkout button may appear correctly in Chrome but overlap with another element in Safari. Similarly, a payment pop-up may work in one browser but fail in another. Systematic cross-browser testing can identify these issues before release.

Tools such as Selenium, Playwright, Cypress, BrowserStack, and LambdaTest can be used for testing. Responsive testing, visual regression testing, and automated testing can provide better coverage.

Thus, a systematic cross-browser testing strategy improves compatibility, reliability, user experience, and consistency of the e-commerce application.

9. A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Answer:

Incorrect salary calculations caused by mistakes in program logic can be classified mainly as functional or logic defects. These defects are generally introduced during the coding phase, although incorrect requirements can also contribute to them.

Defect classification helps testers understand the nature and source of a problem and select suitable testing techniques. For logic defects, techniques such as decision table testing, statement coverage, branch coverage, and condition coverage can be used.

In a payroll system, important calculations such as basic salary, HRA, overtime, tax, and deductions should be clearly defined and tested. Different salary values, tax conditions, overtime hours, and employee categories should be included in the test data.

For example, if the salary is calculated using the formula $\text{basic salary} + \text{HRA} + \text{overtime} - \text{tax} - \text{PF}$, the tester can independently

calculate the expected salary and compare it with the application's result. This can reveal incorrect calculation logic.

To prevent such defects, the team should conduct code reviews, perform unit testing for calculation functions, use proper currency data types, and maintain automated regression tests for previously identified defects.

Therefore, proper defect classification combined with black-box and white-box testing helps detect logical errors early and improves the accuracy and reliability of the payroll system.

10. A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Answer:

Random testing does not provide proper coverage and may miss important defects. Therefore, testing should be treated as an engineering activity with a planned, systematic, and measurable process.

First, the team should analyse the requirements and identify the conditions that need to be tested. Test cases should then be prepared with a test ID, objective, preconditions, input data, expected result, and priority. Each test case should be reviewed and connected to its related requirement.

Structured test case design is important because it ensures that valid, invalid, boundary, and important business conditions are covered. It also makes testing repeatable and allows the same tests to be executed during future releases.

For example, if a password must contain between 8 and 12 characters, testing only normal passwords may miss an error. The tester should check values such as 7, 8, 12, and 13 characters. This can identify incorrect boundary conditions in the program.

Techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing, statement and branch coverage, exploratory testing, and risk-based prioritisation can improve test effectiveness.

Therefore, structured test case design provides better coverage, improves defect detection, saves testing time, and makes the overall testing process more reliable and repeatable.

11. A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Answer:

Data consistency is very important in a healthcare application because patient information such as personal details, prescriptions, laboratory reports, and billing information is shared between different modules. Incorrect data can affect patient safety and the quality of treatment.

The problem can be analysed using testing principles such as early testing and defect clustering. Testers should check important interfaces where data moves between modules and give higher priority to critical patient information.

Defects may originate because requirements do not clearly define a single source of data, different modules store duplicate information, incorrect data types or formats are used, or APIs do not map information correctly. Incomplete transactions, concurrent updates, and migration problems can also cause inconsistency.

For example, if a patient's weight is changed from 70 kg to 65 kg in the Nursing module, systematic testing should verify that the Pharmacy, Billing, and Patient Summary modules also display 65 kg correctly.

Defect reports should clearly mention the affected module, data item, severity, patient-safety impact, environment, and supporting evidence such as screenshots or logs. A defect tracking tool can be used to maintain proper status and traceability.

Regular defect reviews, root-cause analysis, and regression tests for previously identified data problems can further improve data integrity.

Therefore, systematic testing and proper defect reporting help maintain consistent patient data, reduce serious errors, and improve the reliability of the healthcare application.

12. A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Answer:

Repeated defects indicate that the testing team is not effectively using information from previous releases. A defect repository is a central collection of information about defects, including their type, affected module, severity, root cause, status, and fix details.

The repository should first be analysed to identify duplicate and similar defects from earlier releases. The team should also check whether the actual root cause was corrected or only the visible problem was fixed. Reopened defects and defects that escaped into production should receive special attention.

Repository data helps in designing better test cases. If a particular module has many previous defects, it should receive more testing. Similarly, if many defects are related to boundary

conditions, testers should include more Boundary Value Analysis tests. Previous production defects can also be converted into regression test cases.

Defect patterns can be identified using methods such as Pareto analysis, trend analysis, defect density, and root-cause analysis. For example, if input validation defects repeatedly occur across releases, input validation should become a major focus during future testing.

To prevent recurring defects, every defect should have a proper root cause and phase of introduction. Fixed defects should be added to automated regression testing. Regular defect analysis meetings, coding guidelines, review checklists, and developer training can also be introduced.

Therefore, effective use of a defect repository helps the organization learn from previous failures, improve test design, reduce repeated defects, and produce more reliable software.

13. A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Answer:

The failure of a web-based learning system during peak usage indicates that the system was mainly tested for normal functional conditions and not for heavy user loads. Important performance-related scenarios were not properly included in the testing process.

The first step is to identify the missing testing areas. The team should perform load testing, stress testing, spike testing, and endurance testing. Peak situations such as examination registration, assignment submission deadlines, and simultaneous video access should also be considered.

Test planning should define clear performance requirements such as the expected number of concurrent users, acceptable response time, and maximum error rate. Test coverage should include important user scenarios, different load levels, and realistic system configurations.

For example, during a load test, if 2,500 users access the system simultaneously and the login response time increases to 20 seconds, the tester may identify a database connection pool or query performance problem. After improving the database configuration and queries, the system can be tested again with a higher number of users.

Tools such as JMeter, Gatling, or LoadRunner can be used for performance testing. Monitoring tools can also help identify CPU, memory, database, and network bottlenecks.

Therefore, proper performance planning, realistic test conditions, adequate coverage, and regular load testing can identify weaknesses before deployment and improve the reliability and scalability of the learning system.

14. A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Answer:

Testers should be involved from the early stages of the Software Development Life Cycle rather than starting testing only after coding is completed. This approach is known as early testing or shift-left testing.

During the requirements stage, testers can review requirements and identify unclear, incomplete, or incorrect conditions. During the design stage, they can identify possible risks and prepare test scenarios. Test planning and test case preparation can also be started while development is taking place.

Early involvement improves quality because defects in requirements and design can be identified before they reach the coding stage. Fixing defects early usually requires less time and effort. It also helps testers understand the system better and prepare suitable test cases in advance. For example, consider an online food-ordering application with a requirement stating

that a discount is given for orders “above ₹500.” A tester may ask whether an order of exactly ₹500 should receive the discount and whether delivery charges are included. Clarifying this before development prevents incorrect implementation.

Team collaboration can be improved by involving testers in requirement meetings, design discussions, sprint planning, and daily meetings. Developers and testers can work together through reviews, defect discussions, and shared testing activities.

Therefore, early involvement of testers improves communication, reduces rework, detects defects earlier, and helps deliver higher-quality software.

15. A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Answer:

Incomplete test coverage means that some important requirements, inputs, conditions, or program paths have not been tested. Therefore, the first step is to identify which areas are missing from the existing test cases.

The tester should compare the requirements with the executed test cases using a Requirement Traceability Matrix. Code coverage tools can also be used to identify untested statements, branches, and conditions. Boundary values, invalid inputs, and different combinations of inputs should also be checked.

Proper test coverage improves reliability because more requirements and possible execution paths are verified. It also helps identify hidden defects before the software is released and provides better confidence during regression testing.

For example, consider a grading function where 90 and above is A, 75–89 is B, 50–74 is C, and below 50 is Fail. If the team tests only 95, 80, and 60, it may miss a boundary error at 75. Therefore, values such as 49, 50, 74, 75, 89, and 90 should also be tested.

Techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing, statement coverage, branch coverage, condition coverage, and pairwise testing can improve validation.

Thus, proper test coverage and systematic test case design help identify gaps, detect more defects, and improve the reliability of the software.

16. A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Answer:

Poor defect reporting can delay the identification and correction of problems in a logistics application. A clear and standard reporting process should therefore be followed for every defect.

The tester should clearly distinguish between a failure, defect, and error. The failure describes the incorrect behaviour observed in the application, while the defect refers to the fault causing that behaviour. This helps the development team understand and investigate the issue correctly.

A structured defect report should contain the defect ID, title, module, environment, build version, steps to reproduce, expected result, actual result, severity, priority, and supporting screenshots or logs.

For example, instead of reporting “Tracking not working,” the tester should write the exact steps used to reproduce the problem and mention what was expected and what actually happened. This allows the developer to reproduce the issue without repeatedly asking for additional information.

A proper defect life cycle such as New, Assigned, Fixed, Retest, Closed, or Reopened should also be followed. A defect tracking tool can be used to maintain status and ownership.

Reporting standards can be improved by using a common template, defining severity and priority clearly, reviewing reports before submission, and conducting regular defect-triage meetings.

Thus, clear definitions and structured defect reports improve communication, reduce investigation time, make defect tracking easier, and help the team resolve problems faster.

17. A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Answer:

When different teams test different modules using different methods, the testing results and quality may vary. A process-based testing approach can solve this problem by establishing one common testing process for all modules.

The organization should define standard stages such as test planning, test analysis and design, test implementation, test execution, defect reporting, and test closure. Common templates should be used for test plans, test cases, defect reports, and test summary reports.

The same entry and exit criteria should be applied to every module. Roles and responsibilities should also be clearly defined so that each testing activity has an assigned person.

For example, consider an application containing Billing, Inventory, and Reports modules. If Billing is tested in detail while Inventory uses only an informal checklist, testing quality will be inconsistent. By using the same test case format, test design techniques, coverage requirements, and defect reporting process for all three modules, testing becomes uniform.

Consistency can be maintained through peer reviews, common testing tools, regular process audits, team training, shared automation standards, and monitoring of metrics such as test coverage and defect density.

Thus, standardized testing processes reduce gaps, make results easier to compare, improve teamwork, and provide consistent quality across all modules.

18. A system fails after deployment due to untested edge cases. Apply testing principles to

identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Answer:

Edge cases are inputs or conditions that occur at the extreme limits of the expected operating range. They are important because many defects occur near minimum and maximum values, but they are often missed when testing is informal.

Edge cases may be missed because testers use only normal values, requirements do not clearly define limits, testing starts late, or the same test cases are repeatedly used without review. Exhaustive testing is also not practical, so suitable test design techniques are required.

Boundary Value Analysis (BVA) is useful for finding such defects. It tests values at and around the boundaries because programming mistakes commonly occur with conditions such as $<$, $<=$, $>$, and $>=$.

For example, if an age field accepts values from 18 to 60, the tester should check 17, 18, 19, 59, 60, and 61. Invalid values such as negative numbers, empty input, and very large values should also be tested.

Other important edge cases include zero values, special characters, null values, leap-year dates, maximum file sizes, and maximum numbers of items.

To improve test design, the team should clearly define input limits, use Equivalence Partitioning with Boundary Value Analysis, include negative tests, maintain edge-case checklists, review test cases regularly, and add production edge-case defects to regression testing.

Thus, systematic boundary and edge-case testing helps identify hidden defects before deployment and improves the reliability of the system.

19. A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves

defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Answer:

A defect repository is a central and organised place where all software defects are recorded and managed. Without proper documentation, defects may be lost in emails or messages, duplicate defects may be created, and their status may become difficult to track.

The company should introduce a defect tracking tool such as Jira, Bugzilla, or Azure DevOps as a common repository. Each defect should contain important details such as defect ID, title, description, affected module, environment, steps to reproduce, expected result, actual result, severity, priority, status, root cause, and fix details.

Proper documentation helps developers reproduce and fix defects faster. It also makes ownership and status visible and helps managers understand the quality of the software. Previous defects can be searched easily, which helps identify duplicate or similar problems.

A structured repository also supports defect analysis. The team can study defect types, affected modules, severity, root causes, and recurring problems. This information can be used to improve test cases and identify areas that require additional testing.

For example, if several defects are repeatedly found in the payment module, the testing team can analyse the repository and introduce additional payment-related test cases and regression tests.

Documentation can be improved by using a standard defect template, requiring proper reproduction steps and evidence, searching for duplicates before creating new defects, updating status regularly, and recording the root cause when the defect is closed.

Thus, a properly maintained defect repository improves defect tracking, communication, analysis, and prevention of recurring problems, resulting in better software quality.

20. A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality.

Answer:

Repeated transaction failures indicate that the banking system is not being tested in a systematic manner. The testing process should be redesigned using proper planning, test design, execution, defect reporting, regression testing, and measurement.

First, the team should identify all important transaction types such as deposit, withdrawal, fund transfer, and bill payment. Test cases should cover valid inputs, invalid inputs, boundary conditions, transaction states, and failure situations. Testing should also be risk-based, giving higher priority to critical money-related operations.

Important banking properties such as **Atomicity, Consistency, Isolation, and Durability (ACID)** should be tested. For example, if ₹1,000 is transferred from one account to another and the system fails after debiting the first account, the transaction should be rolled back so that money is not lost.

Boundary Value Analysis can also be used. If the daily transaction limit is ₹1,00,000, the tester should check ₹99,999, ₹1,00,000, and ₹1,00,001.

Concurrency testing should verify that simultaneous transactions do not produce an incorrect balance. Duplicate transaction testing should also ensure that clicking the payment button twice does not result in two debits.

Database testing, API testing, security testing, performance testing, and failure-recovery testing should be included. Automated regression testing should be performed after every important change so that previously fixed transaction defects do not return.

Therefore, structured and risk-based testing helps detect transaction failures earlier, prevents repeated defects, protects financial data, and improves the reliability and trustworthiness of the banking system.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand